

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	K.KEERTHANA SAI	Reg. No.:	192425143
Section / Batch:	SLOT-D/2024	Date:	

Code of Conduct

I K.Keerthana Sai (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

ANSWERS:

Q1. Reference Resolution Using Multiple Constraints

Explanation

Reference resolution means finding which entity a pronoun refers to.

In the given sentence:

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

The pronouns are:

- she
- her
- She
- her
- it

The algorithm considers:

1. Gender
2. Number
3. Recency
4. Semantic compatibility

Both Meena and Kavya are female and singular, so gender and number alone are not enough. Therefore, context and semantic meaning are also considered.

Pseudocode

ALGORITHM ReferenceResolution

INPUT: Discourse

OUTPUT: Resolved discourse

1. Identify all entities and pronouns.
2. For each pronoun:
 - Find possible antecedents.
3. Check gender compatibility.
4. Check number compatibility.
5. Check recency.
6. Check semantic compatibility.
7. Calculate the score of each candidate.
8. Select the candidate with highest score.
9. Replace the pronoun with the selected antecedent.
10. Return resolved discourse.

END

OUTPUT:

```
C: > Users > keert > COS-AT2.1.py >...
1 text = ""Meena gave Kavya a laptop because she needed one
2 for her project. She thanked her and started using it.""
3
4 entities = ["Meena", "Kavya", "laptop"]
5
6 print("Entities:", entities)
7
8 pronouns = ["she", "her", "She", "her", "it"] Generate code: Alt+Sh
9 print("Pronouns:", pronouns)
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\keert\AppData\Local\Programs\Microsoft VS Code> & C:\Users\keert\AppData\Local\Programs\Microsoft VS Code>
Entities: ['Meena', 'Kavya', 'laptop']
Pronouns: ['she', 'her', 'She', 'her', 'it']

Reference Resolution:
she -> Kavya
her (project) -> Kavya
She -> Kavya
her (thanked) -> Meena
it -> laptop

Resolved Discourse:
Meena gave Kavya a laptop because Kavya needed one
for Kavya's project. Kavya thanked Meena and started using the laptop.
```

Final Answer

The algorithm successfully resolves the pronouns by considering gender, number, recency and semantic compatibility. Semantic context helps distinguish between Meena and Kavya when grammatical features are identical. Hence, incorrect antecedent selection is reduced.

Q2. Entity Chains and Discourse Coherence

Explanation

An **entity chain** is a sequence of words or expressions that refer to the same entity.

Given:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

The two major entity chains are:

Person Chain

Anjali → She → Anjali

Laptop Chain

laptop → The laptop → it → the computer

Because these entities continue across sentences, the paragraph is coherent.

Pseudocode

ALGORITHM EntityChainAnalysis

INPUT: Discourse

OUTPUT: Entity chains and coherence

1. Divide text into sentences.
2. Extract entities and referring expressions.
3. Compare expressions using lexical and semantic similarity.
4. Link expressions referring to the same entity.
5. Construct entity chains.
6. Measure entity continuity across sentences.
7. If important entities continue across sentences:
 Mark discourse as COHERENT.
Else:
 Mark discourse as LESS COHERENT.
8. Display entity chains and result.

END

OUTPUT:

```
C:\Users\keert > CO5-AT2.2.py > ...
1 sentences = [
2     "Anjali bought a new laptop.",
3     "The laptop had a powerful processor.",
4     "She installed Python on it.",      Generate code: Alt+Shift+Enter
5     "Later, Anjali used the computer to develop a machine learning application."
6 ]
7
8 person_chain = ["Anjali", "She", "Anjali"]
9 laptop_chain = ["laptop", "The laptop", "it", "the computer"]
10
11 print("Person Entity Chain:")
12 print(" -> ".join(person_chain))
13
14 print("\nLaptop Entity Chain:")
...

Problems Output Debug Console Terminal Ports
PS C:\Users\keert\AppData\Local\Programs\Microsoft VS Code> & C:\Users\keert\AppData\Local\...
Person Entity Chain:
Anjali -> She -> Anjali

Laptop Entity Chain:
laptop -> The laptop -> it -> the computer

Entity Continuity: 1.0
Coherence: COHERENT
```

Final Answer

The discourse is **coherent** because the main entities, Anjali and the laptop, are continuously maintained through referring expressions. If the entity chain were broken, the reader could lose track of the topic, thereby reducing discourse coherence.

Q3. Algorithm for Identifying Discourse Relations

Explanation

Discourse relation identifies the **logical relationship between sentences**.

Given:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

The relations are:

- Server crashed → users couldn't access → **Cause–Effect**
- Access problem → server restarted → **Problem–Solution**
- Server restarted → system available → **Sequence**

Important discourse markers are:

- **As a result** → Cause–Effect
- **After that** → Sequence

Pseudocode

ALGORITHM DiscourseRelation

INPUT: Discourse

OUTPUT: Relations between sentences

1. Divide discourse into sentences.
2. Identify discourse markers.
3. For each consecutive pair:
 Analyze contextual meaning.
4. If marker indicates a consequence:
 Assign Cause-Effect.
5. If one sentence solves a problem:
 Assign Problem-Solution.
6. If marker indicates order:
 Assign Sequence.
7. Construct discourse structure.
8. Return relations.

END

OUTPUT:

```
C:\Users> keert > COS-AI2.3.py > ...  
1 sentences = [  
2     "The server crashed unexpectedly.",  
3     "As a result, users could not access the application.",  
4     "The technical team restarted the server.",  
5     "After that, the system became available again."  
6 ]  
7  
8 relations = [  
9     ("S1", "S2", "Cause-Effect"),  
10    ("S2", "S3", "Problem-Solution"),  
11    ("S3", "S4", "Sequence")  
12 ]  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100
```

Problems Output Debug Console Terminal Ports

Discourse Relations:

```
S1 -> S2 : Cause-Effect  
S2 -> S3 : Problem-Solution  
S3 -> S4 : Sequence
```

Discourse Structure:

```
Server Crash  
↓ Cause-Effect  
Users Cannot Access Application  
↓ Problem-Solution  
Server Restarted  
↓ Sequence  
System Available
```

Final Answer

The discourse is logically coherent because the events form a meaningful chain:

Problem → Effect → Solution → Result

The markers “**As a result**” and “**After that**” provide explicit evidence for the identified relations.

Q4. Algorithm for Dialogue Act Classification

Explanation

A **dialogue act** represents the purpose or intention of an utterance.

For example:

- “Hello” → Greeting
- “I need help” → Request
- “What problem are you facing?” → Question
- “I do not understand machine learning.” → Inform
- “I can explain...” → Offer

The system identifies keywords and sentence patterns to determine the dialogue act.

Pseudocode

ALGORITHM DialogueActClassification

INPUT: Conversation

OUTPUT: Dialogue act of each utterance

1. Read each utterance.
2. Convert text to lowercase.
3. Extract keywords and sentence patterns.
4. If greeting word exists:
Assign Greeting.
5. Else if request expression exists:
Assign Request.
6. Else if question pattern exists:
Assign Question.
7. Else if offer expression exists:
Assign Offer.
8. Else:
Assign Inform.
9. Store the dialogue act.
10. Return dialogue-act sequence.

END

OUTPUT:

```
C: > Users > keert > COS-AT2.4.py > classify

1 conversation = [
2     ("User", "Hello, I need help with my assignment."),
3     ("Agent", "Sure, what problem are you facing?"),
4     ("User", "I do not understand machine learning."),
5     ("Agent", "I can explain the basic concepts to you.")
6 ]
7
8 def classify(text):
9     text = text.lower()    Generate code: Alt+Shift+Enter
10
11     if "hello" in text or "hi" in text:
```

Final Answer

Dialogue-act recognition helps a conversational agent understand the **user's communicative intention**. It enables the system to choose an appropriate response, such as answering a question, fulfilling a request, or offering assistance.

Q5. Dialogue State Tracking

Explanation

Dialogue State Tracking maintains information collected during a multi-turn conversation.

The required slots are:

- Departure City
- Destination City
- Travel Date
- Number of Passengers

Conversation:

User: I want to book a flight.

Agent: Where would you like to travel?

User: I want to go to Delhi.

Agent: From which city?

User: From Chennai.

After processing the conversation:

Destination = Delhi

Departure = Chennai

Travel Date = Unknown

Passengers = Unknown

The next question should therefore be:

“What is your travel date?”

Pseudocode

ALGORITHM DialogueStateTracking

INPUT: User conversation

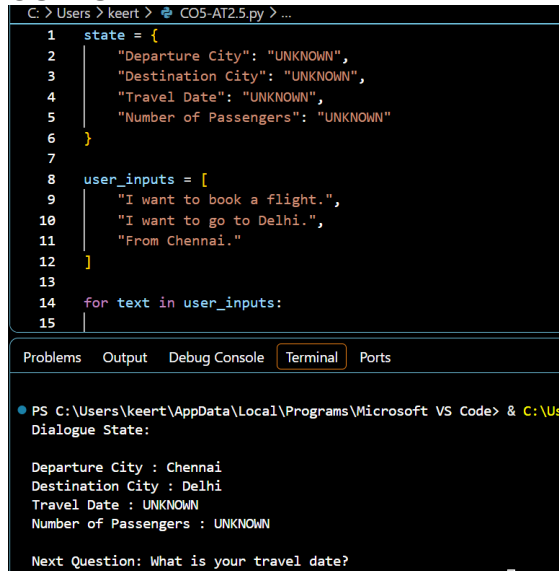
OUTPUT: Updated dialogue state and next question

1. Initialize all slots as UNKNOWN.
2. Read each user utterance.
3. Extract destination city.
4. Extract departure city.
5. Extract travel date.

6. Extract number of passengers.
7. Update the corresponding slots.
8. Find missing slots.
9. Ask a question for the next missing slot.
10. If no slot is missing:
 Confirm all booking details.
11. Return dialogue state.

END

OUTPUT



```
C:\Users\keert > COS-AT2.5.py > ...  
1 state = {  
2     "Departure City": "UNKNOWN",  
3     "Destination City": "UNKNOWN",  
4     "Travel Date": "UNKNOWN",  
5     "Number of Passengers": "UNKNOWN"  
6 }  
7  
8 user_inputs = [  
9     "I want to book a flight.",  
10    "I want to go to Delhi.",  
11    "From Chennai."  
12 ]  
13  
14 for text in user_inputs:  
15  
  
Problems Output Debug Console Terminal Ports  
  
PS C:\Users\keert\AppData\Local\Programs\Microsoft VS Code> & C:\Us  
Dialogue State:  
  
Departure City : Chennai  
Destination City : Delhi  
Travel Date : UNKNOWN  
Number of Passengers : UNKNOWN  
  
Next Question: What is your travel date?
```

Final Answer

Dialogue State Tracking successfully stores information from previous turns and identifies missing information. Therefore, the agent does not repeatedly ask for information already provided and can continue the conversation coherently.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	4	3	4	4	19	97
2	4	4	4	4	4	20	
3	4	3	4	4	4	19	
4	4	3	4	4	4	19	
5	4	4	4	4	4	20	

Faculty In-charge	Course Coordinator	Program Director
Dr.Kumaragurubaran.T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____